Masaryk University
Faculty of Informatics



Integration of SHA2 FPGA implementation with RISC-V core

Master Thesis

Rajesh Chandrakant Mehta

Bangalore, December 2017

Declaration

Hereby I declare, that this thesis is my original authorial work, which I have worked out by my own. All sources, references and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Rajesh Chandrakant Mehta

Advisor: prof. Ing Václav Prenosil

Acknowledgement

I would like to thank Govt. of India for their support to achieve my degree from Masaryk University.

My sincere thanks to my supervisor prof. Ing. Václav Prenosil, C.Sc. and field guarantor prof. RNDr. Václav Matyáš, M.Sc., Ph.D., for their co-operation and guidance.

I would like to appreciate the support from my colleagues.

Further, I would like to extend my thanks to our technical mentor Dr. Subrata Rakshit from DRDO and Dr. V Kamakoti from IIT, Chennai for their precious comments and advices. I appreciate the most helpful support from RISE lab team members, IIT, Chennai.

The authors, co-authors, whose papers I have referred for carrying out my thesis, many thanks to them.

Last, I would like to thanks my family, Hardi, my wife, Nandini my princess and my parents for their patience and encouragement.

Abstract

In modern days, most of the developments are preferred over free open source. In this direction, academia has initiated the work in free open source Instruction Set Architecture (ISA) for design and development of processor. Such an activity is progressing at IIT, Chennai under the name of SHAKTI [1] with the support from UC, Berkeley. It is based on RISC V Core. The objective of this thesis is to integrate the SHA-2 [2] FPGA implementation with SHAKTI as one of the peripherals in SoC format.

The theoretical part of thesis is to carry out extensive survey of the literatures [3, 4, 5] for SHA-2 implementation. HDL language such as Bluespec System Verilog (BSV) [6] is utilized for this implementation.

Subsequent to the implementation, thesis includes the identification of best integration strategy of the SHA-2 design with the host processor and evaluation of the same based on area, power and performance overheads.

Keywords

RISC- V Core, Secure Hash Algorithm-2, Bluespec System Verilog, SHAKTI C-64, High Level Synthesis Language, Field Programmable Gate Arrays.

Contents

1 **Introduction** 1
  1.1 Secure Hash Algorithm (SHA) [9] 2
  1.2 RISC - V Core, SHAKTI 3
  1.3 Bluespec System Verilog (BSV) 4
2 **Related work** 6
  2.1 Literature Survey 6
  2.2 Summary of Literature Survey 9
3 **Implementation of SHA-2** 10
  3.1 High level block diagram 11
  3.2 Functional block diagram 12
  3.3 Comparative Analysis 17
4 **Integration with RISC-V core** 19
  4.1 Description of identified strategy 20
  4.2 Result 22
5 **Conclusion** 23

List of Tables

3.1 Synthesis result on Virtex-7 Family *17*
3.2 Synthesis result on Artix-7 Evaluation platform *18*
3.3 Synthesis result on Stratix-V Family *18*
4.1 Synthesis result using RTL Compiler *22*

List of Figures

2.1 Proposed architecture described in paper[3] *6*
2.2 Proposed architecture described in paper[4] *7*
2.3 Block diagram of SHA-256 interfacing with PC described in paper [5] *8*
3.1 High level block diagram of SHA-2 implementation by thesis *12*
3.2 Functional block diagram of SHA-256 implementation by thesis *13*
3.3 Implementation of Round Computation in BSV carried out in thesis *16*
4.1 Identified strategy for integration with RISC V core *19*

List of Abbreviation

ISA	Instruction Set Architecture
UC	University of California
SHA-2	Secure Hash Algorithm-2
BSV	Bluespec System Verilog
NIST	National Institute of Standards and Technology
NSA	National Security Agency
RISC	Reduced Instruction Set Computer
CISC	Complex Instruction Set Computer
SoC	System on Chip
HDL	Hardware Design Language
CLB	Control Logic Block
FPGA	Field Programmable Gate Array
ASIC	Application Specific Integrated Circuit
FSM	Finite State Machine
ALM	Adaptive Logic Module
FF	Flip Flop
AXI	Advanced eXtensible Interface
RC	Encounter RTL Compiler

List of Notation

The following notations are used in defining the SHA-2 algorithm.

H^i	i^{th} hash value.
a,b,c,d,...,h	working variables.
H^i_j	j^{th} word of ith hash value.
K^t	constant values used in t^{th} iteration.
k	number of padded zeros.
l	incoming message length in bits.
M	incoming message.
N	number of blocks.
T_1, T_2	temporary variable.
W_t	t^{th} word of message schedule.

Chapter 1: Introduction

In the current days, most of the developers, researchers want to utilize open source codes. This is because of its' greatest advantage that it is free use. Further, one can modify and distribute. This result into low cost because proprietary parts only increase the cost parts, in most cases. Subsequent to this, as per article[7], open source codes are more secured compare to proprietary one, as users need not to wait for plug-ins. Open community will help in exploring and fixing the bugs. Further, it is continuous process with large community. In the same direction, researchers have started analyzing the open source for Instruction Set Architecture(ISA). Till the time, developers were using the processors from Intel, ARM and many more however, their ISA codes were never been shared or open source. This might be because of business/marketing strategy or something else.

The University of California(UC) at Berkeley has initiated the activity for open ISA[8]. The activity is to make open ISA for RISC V Core. At India, Indian Institute of Technology, Chennai has joined to progress this activity. The objective of team is to realize processor with open ISA. The name of this processor is SHAKTI.

Thesis aim is to integrate the security algorithm with SHAKTI in SoC format, which can be seen as one of power of open source ISA. In that regard, thesis has selected the Secure Hash Algorithm-2 (SHA-2)[2] because of its popularity, well-implemented and well-run. Further, the code of SHAKTI is written into BlueSpec System Verilog (BSV)[6], thesis implements the SHA-2 in BSV.

The thesis also reviews related works and finding the best strategy for integration with SHAKTI C-64 core. It also carries out the evaluation of the same based on area, power and performance overheads.

1.1 Secure Hash Algorithm (SHA) [9]

The family of cryptographic hash function contains secure hash algorithm, designed by National Institute of Standards and Technology (NIST). This list includes SHA-1, SHA-224, SHA-256, SHA-384 nd SHA-512. By latest, SHA-3 also included into list.

This secure hash function maps any arbitrary length message to n-bit hash values. This n-bit cryptographic hash has following properties:

- *pre-image*: with given hash value, it is quite difficult to get back original message from hash bits.
- *second pre-image*: with given message m_1, it can be seen that there is less probability to find the message m_2 with same hash bits.
- *collision resistant*: The property says that it is less probable that two different messages have same hash bits.

Further, the avalanche effect ensures that two different messages, even when they differ in single bit, there hash values are quite far with very high probability.

The above characteristics enable the utilization of hash function in some of the wide range of its applications as shown below:

- *Password validation*: secure implementation of security module does not allow the original password to be stored in RAM. It is better to store the digest. When the user enters the password, digest is calculated on-line and compared with stored hash value. The pre-image property of hash function, protect the original password.
- *Authentication of Challenge handshake*: Secure communication over not-trusted channel can be achieved by transmitting only digest shared secret.
- *Tamper proof* : The official software of secure product can be declared as tampered if its hash does not match with published hash by official developer.

- *Digital Sign*: The hash functions are commonly used with digital signature with encryption.

In 2005, after finding the collision in SHA-1, with 2500 fewer steps which degraded the usage of SHA-1 in secure application. SHA-2 shows significant improvement from its predecessors SHA-1 which was designed by National Security Agency (NSA).

The variants of SHA-2 has nomenclature from the resulted length of digest bits. SHA-224, 256, 384 and 512 have respectively digest length as 224, 256, 384 and 512.

The SHA-2 algorithm has been thoroughly analyzed over the year since it was published. Further, well-written published related works can be found easily. In addition, the code of SHA-2 is well implemented with well run, this will accelerate its' implementation in new language called Bluespec System Verilog, which thesis proposes to do.

The thesis describes algorithm of SHA-2 in-detail along with its implementation in subsequent chapter.

1.2 RISC - V Core, SHAKTI

We are aware about the advantages of Reduced Instruction Set computer (RISC) over Complex Instruction Set computer (CISC) and with that our modern electronic equipments are growing. In continuation of this, UC Berkeley has initiated research in design of next generation of processors. For that, RISC -V Core based Instruction Set Architecture (ISA) is proposed.

The work for designing and developing the processor based on this core, have been progressed by Indian Institute of Technology, Chennai,India [1]. The processor is named as SHAKTI.

SHAKTI processor has following variants: C Class microcontrollers, I Class processors, M Class processors, S class, H class and T class processor. These variants have been derived to support heterogeneous applications/devices like Low power, Mobile, Server, OS, Compilers, BSP and related software sub-systems will be part of this development.

The aim of research team is to design and develop the system on chip (SoC)with open source code. Further, the interconnects and stor-

age system will also kept in mind.

The thesis is focusing only on Crypto class microcontroller, namely C Class. Integration is carried out with this platform. C class microcontrollers are

- 32 bit 3-8 stage aimed at 50-250 MHz.
- it will be featured with very low power static design.
- it has optional memory protection and also support for ISO26262 applications for Fault tolerant requirement.

The RISC V Core, such as C-64, is selected for integration because it is evolving processor under open source theme. Further, the entire code and ISA is license under Berkeley and it is open standard so that interested can edit the ISA as per their requirements.

1.3 Bluespec System Verilog (BSV)

Bluespec System Verilog is the new Hardware Design Language (HDL) based upon IEEE standard hardware verilog. The different stages of hardware design includes architecture, design, verification and testing and finally physical design. Huge impact on these stages has been witnessed with BSV. The main advantages of BSV are [10]:

- BSV has familiar module hierarchy as follows in Verilog. This helps developers, who already know Verilog.
- *Atomic Transactions*: The developer free from *always blocks*. The BSV has in-built control logic for accessing concurrent and complex shared resources.
- *Atomic Methods*: BSV replaces *port lists* of Verilog, which helps in removing bugs related to interfaces and race conditions.
- *Types and Type-checking*: BSV provides more powerful types and strong type checking. This ensures the discipline in clock domain.

- The designers and developers have powerful parameterization with BSV compare to Verilog. Micro-architectures' range can be parameterized.
- With less time, the developers can developed models, designs and subsequent test-benches, too.
- The best advantage of BSV is that it is fully synthesizable at all levels of design.

With above benefits from BSV, increase in the speed of designing the System On Chip (SoC) compare to existing HDLs is quite obvious. Because of above advantages of BSV and the code of RISC V core is also written in BSV, the selection of Bluespec System Verilog becomes obvious.
The organization of the thesis is as follows:
Chapter - I Introduction of SHA-2, RISC V Core such as SHAKTI C-64 and Bluespec System Verilog (BSV).
Chapter - II Related work.
Chapter - III Implementation of SHA-2.
Chapter - IV Integration with RISC-V Core
and finally *Chapter - V* conclude the thesis.

Chapter 2: Related work

2.1 Literature Survey

Many researchers from academia and industry have been working on implementation of SHA algorithms of various hardware platforms like FPGAs, ASICs and SoCs. Thesis had surveyed required all articles and projected here in this chapter.

The paper [3] proposed following architecture for hardware implementation of SHA-2. This paper was published in very less time after publish of SHA-2 therefore very naive but effective approach was followed.

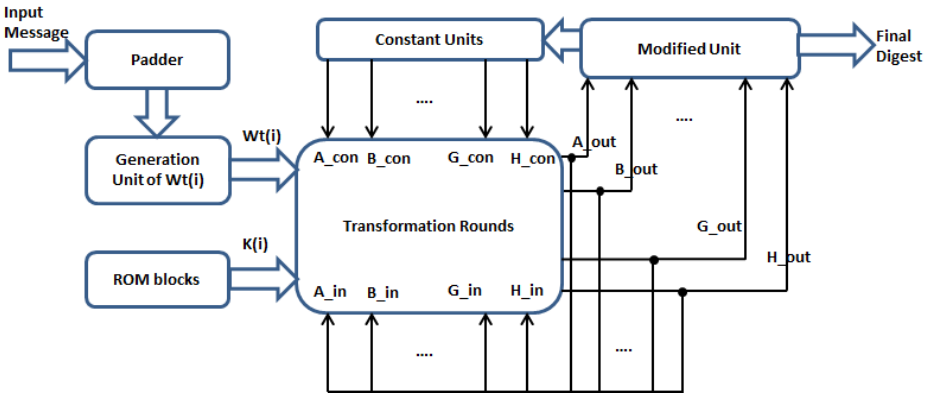


Figure 2.1: Proposed architecture described in paper[3]

The figure 2.1 shows various modules required in computing SHA-2 digest. As the name suggest, padder module pads the incoming message such that length will be multiple of 512bits for SHA-256 and 1024 bits for SHA-512. The generator module takes care of W(t)- mes-

sage schedule from padded incoming message. The ROM blocks, Constant units are consist of the constant values of K(t) and $H^{(0)}$. The constant units have the implementation of constant functions of SHA2 like ch(x,y,z), maj(x,y,z), $\sum_0(x)$, $\sum_1(x)$, $\sigma_0(x)$ and $\sigma_1(x)$. Main computation block represent the transformation round for t = 0 to t = 64 for updating of a,b,...,h for each N. Further, these a,b,...,h update the $H^{(i)}$ and result into final digest.

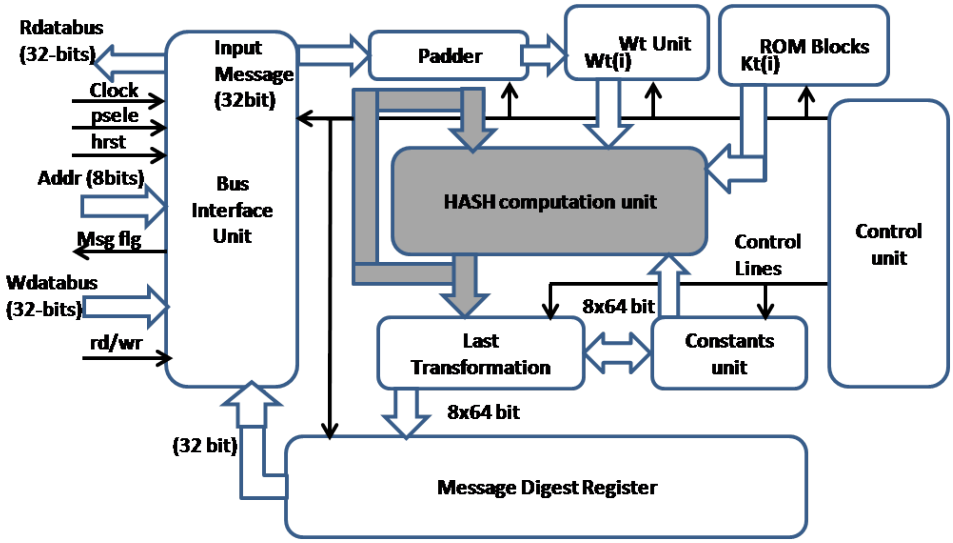


Figure 2.2: Proposed architecture described in paper[4]

The proposed architecture explained in paper[3] was implemented using VHDL, HDL language and executed on Virtex family of XILINX FPGA device (v200pq240). The implementation is compared with other implementations, published in [11] with MD5 secure hash. The Control Logic Blocks (CLBs) utilized in proposed architecture was 1060 to 2237 respectively for SHA-256, SHA-384 and SHA-512.

Paper [4] proposed architecture which support multi mode operations for SHA-2 family. Based on user inputs, particular SHA-256, 384 or 512 will be computed. As shown in figure 2.2, the particular operation of SHA-2 will be selected with the help of write command from user during initialization. This is handled by control unit. The ROM blocks and constant units have similar responsibility as described in

paper [3].

The Hash computation unit in above figure does similar activity as described in main computation. But here the number of transformation rounds for different SHA-2 modes will be selected with the support of feedback. Last transformation carries out the final digest. For efficient communication, the bus interface module is derived. The implementation was carried over on Virtex (v200pq240) of Xilinx FPGA device. Power consumption of 49mW was measured with the implementation. It was measured that the proposed solution was compared with area-delay product with separate implementation of SHA-2. This was observed approximate same with individual implementation. The same observation for throughput was recorded.

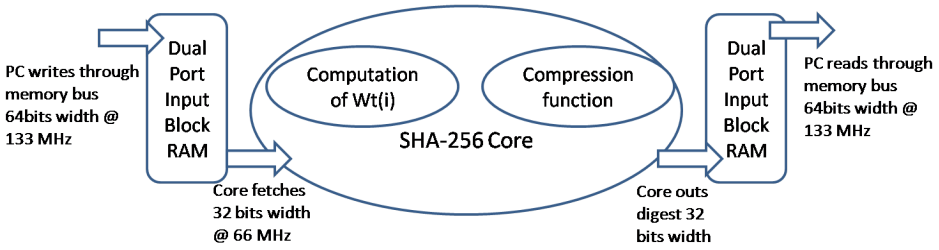


Figure 2.3: Block diagram of SHA-256 interfacing with PC described in paper [5]

Paper [5] has introduced the very novel architecture towards the FPGA compared to Application Specific Integrated Circuits (ASIC) design. The implementation of SHA-256 was carried out with the help of shift register and intermediate hash mechanism, which is described in other related work. The figure 2.3 shows the interface between PC and FPGA consist of SHA-256 design. Here the implementation of SHA-256 is populated on Xilinx Virtex XCV300E-8 FPGA on Pilchard FPGA card. The interconnection between FPGA and pilchard FPGA card [12] is 64 bit wide memory mapped bus. In able to synchronize the transfer of data between PC and card, the dual port Block RAMs are populated on card. PC writes 64 bits data at 133 MHz to input block RAM and 32bit data is read by SHA core at 66 MHz. SHA core computes the digest and writes it to 32 bits output block RAM. Subsequently, through polling mechanism, PC reads the data from

output block RAM at 133 MHz.

The proposed architecture was synthesized and tested on Pilchard platform. It showed that the 512 bits message block takes 8 cycle to load and 65 cycle to process it. The throughput was measured as 87 MB/s for an 88 MHz clock.

2.2 Summary of Literature Survey

The following lessons are learned from literature survey:

- The implementation should be carried out with modular approach since repetition of code can be avoided.
- The implementation should work out the trade off between area and performance. For example, design can use more registers and do the work as fast as possible. At the other side, design can use less registers but takes extra time to carry out processing from optimized number of registers.
- The working environment of design also plays major role in deciding design policy. For example, the implementation needs to work alone with Field Programmable Gate Array (FPGA) i.e. Application Specific Integrated Circuit (ASIC) format or it needs to be integrated with processor i.e. SoC format.

Basic implementation scheme is proposed in paper[3], though, it is very useful for constructing new design policy because of its modular architecture. Subsequently, architecture described in paper[4] accepts the inputs from user to carry out variants of SHA-2 family, i.e. SHA-224, SHA-256, SHA-384 and SHA-512. Further, it has explored much more control interfaces like, write/read, rst. The paper[5] has explored the hardware based interfacing. It advances the knowledge in terms of flow control to interact with SHA-2 FPGA implementation.

Chapter 3: Implementation of SHA-2

First time, Secure Hash Algorithm version 1 was published in 1993 by NIST. With first review in 1995, initial weaknesses were observed and new hashing algorithm was published in 2001. This one is published as Secure Hash Algorithm version 2. Improvements in SHA-1 made, more robust against known attacks, termed as SHA-2. Moreover, it can also be operated on large blocks of data, i,e 2^{128}.

The scope of thesis is to identify the best integration strategy of the SHA-2 design with the host-processor, RISC V core, such as SHAKTI C-64. None of the related work, discussed in previous chapter, can directly adopted for thesis objective because of following reasons:

- *Source code of RISC V*: The available open source code for RISC V Core is in Bluespec System Verilog, new HDL language. The related works were carried out in Verilog or VHDL.

- *Size of Data Bus*: The size of data bus for RISC V Core, Crypto processor, is 64 bits. Therefore, it can handle at a time 64bits data. Implementation of fragmentation-reassembly modules is required to be carry out the integration for larger size of data.

With above points, thesis has proposed the new architecture and carried out implementation. However, thesis has adopted the modular architecture explained in paper[3], the design of interface control proposed in paper[4] and the flow control mechanism explained in paper[5] in proposed architecture. The remaining chapter explains such implementation of SHA-2.

The variants SHA-224 and SHA-384 of SHA-2 are derived from its basic variants SHA-256 and SHA-512 respectively such that the first 224 bits of 256 digest bits of SHA-256 becomes digest of SHA-224 and first 384 bits of 512 digest bits of SHA-512 becomes digest of SHA-384. Further, it was studied and understood from the FIPS document

of SHA-2[13], there are following main differences between SHA-256 and SHA-512:

- *Word Length*: SHA-256 operates on 32 bits wordsize whereas SHA-512 operates on 64 bits word size.

- *Constants*: There are sixty four 32 bits words $K_0^{256}, K_1^{256}, ..., K_{63}^{256}$ in case of SHA-256. In case of SHA-512, there are eighty 64 bits words $K_0^{512}, K_1^{512}, ..., K_{79}^{512}$.

- *Padding Module*: The output of padding for SHA-256 should be multiple of 512 bits whereas for SHA-512, it should be 1024 bits.

- *Message Schedule*: $W_0, W_1....W_{63}$ are the sixty four message schedules with 32 bits word size are heart of SHA-256 whereas W_0, $W_1....W_{79}$ are the eighty message schedules with 64 bits word size are heart of SHA-512.

- *Rounds of computation*: The 64 rounds of computation will result digest of SHA-256 however 80 rounds of computation are required for SHA-512 digest.

Therefore, thesis first describes implementation of SHA-256 and subsequently, above discussed minor modifications will lead to implementation of SHA-512 and further, trimming of final digest bytes of SHA-256 and SHA-512 results into SHA-224 and SHA-384, respectively.

3.1 High level block diagram

The figure 3.1 explains the high level block diagram of SHA-2 implementation, proposed by thesis. Here, the load register passes the input message, whose SHA-2 needs to be computed. The start register is the begin flag for SHA-2 computation. The end of computation will be decided based on status of finish register. Once end of computation is detected, digest can be fetched through fetch register. The above block diagram is designed with focus to achieve second part of thesis, that is the integration with RISC V core.

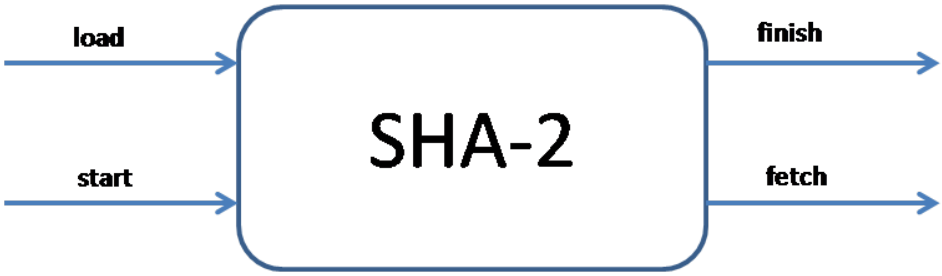


Figure 3.1: High level block diagram of SHA-2 implementation by thesis

3.2 Functional block diagram

The figure 3.2 shows the functional block diagram for SHA-256. The input of figure is padded message of 512 bits and output is intermediate digest value such that it will work any number of padded messages. For example, for N = 1, i.e. the input message length is less than 448 bits and therefore total number of padded message is one. Here intermediate digest is itself is final digest. For N = 2 or more, i.e. the input message length is equal or more than 448 bits and therefore total number of padded message is 2 or more. The intermediate digest is required to push to memory unit to initialize the first hash value with it for second padded message onwards. This design was carried out to make implementation easy and re-usable.

The parsing module maps the 512 bits of padded message to first sixteen W_0, W_1....W_{15} message schedule of 32 bits word size seamlessly. SHA-2 uses following six logical functions, which are expressed by Functions module in figure 3.2.

$$Ch(x,y,z) = (x \wedge y) \oplus (\neg x \wedge z) \tag{3.1}$$

$$Maj(x,y,z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \tag{3.2}$$

$$\sum_{0}^{(256)}(x) = ROTR^{2}(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x) \tag{3.3}$$

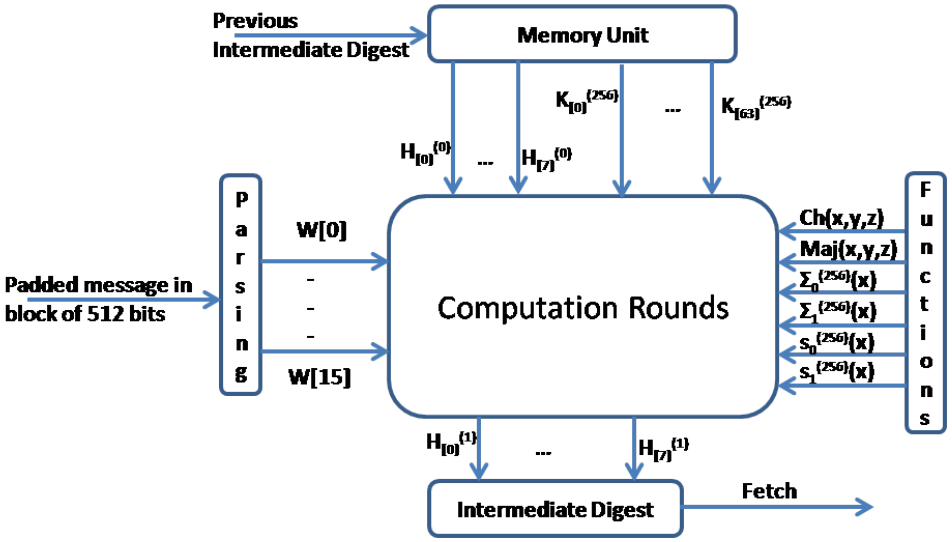


Figure 3.2: Functional block diagram of SHA-256 implementation by thesis

$$\sum_{1}^{(256)}(x) = ROTR^{6}(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x) \tag{3.4}$$

$$s_0^{(256)}(x) = ROTR^{7}(x) \oplus ROTR^{18}(x) \oplus SHR^{3}(x) \tag{3.5}$$

$$s_1^{(256)}(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x) \tag{3.6}$$

Here $\wedge$ is bitwise AND operation on 32 bits, $\oplus$ is 32 bit Ex-OR operation, $\neg$ is NOT operation, ROTR stands for rotate right defined as $ROTR^n(x) = (x \gg n) \vee (x \ll (w - n))$, where w is the bit width of x, $\vee$ is bitwise OR operation and SHR stands for right shift defined as $SHR^n(x) = (x \gg n)$.

The BSV offers advantage for implementation of function. Compare to C language, here, there is no need of switching between stack and user code. Further there is no "call" and "return" from/to current program location. For BSV, it is a just a code piece and have global accessed. They can be plugged into place with its use. Basically, func-

tions, in BSV, are static syntactic convenience and also expanded in line during static elaboration.

Memory unit of the figure initializes sixty four constants with 32 bit words labeled as $K_0^{256}, K_1^{256}, ..., K_{63}^{256}$. These constants are initialized with first 32 bits of fractional parts of the cube roots of the first 64 prime numbers. It is implemented using BSV with the method called RegFile, where BSV will take care of its hardware implementation. This is most advantageous as developer can write the code without worrying about its hardware implementation i.e. ROM or RAM blocks. Whereas, coding in verilog requires developer to separate out the logic for ROM and RAM hardware blocks.

Further, based on previous intermediate digest, it will initialize eight 32 bit initial hash values labeled as $H_0^{(0)}, H_1^{(0)}....H_7^{(0)}$ either with constants, which is first thirty-two bits of the fractional parts of the square roots of the first eight prime numbers or with intermediate digest value of previous block of padded message. Because of advantage of BSV, memory unit has both RAM and ROM block logic.

The round computation module implements following equations:

- *Message Schedule*: For t ϵ [0,15], the message schedule(W_t) is derived using parsing module as discussed above. For t ϵ [16,63], the message schedule(W_t) is derived using :

$$W_t = s_1^{(256)}(W_{t-2}) + W_{t-7} + s_0^{(256)}(W_{t-15}) + W_{t-16} \tag{3.7}$$

  , where $+$ is addition performed over mod 2^{32}.

- *Working and Temporary Variables*: There are eight working variables denoted as a,b,c,d,e,f,g,h and two temporary variables denoted as T_1, T_2. These variables are defined as follows:

$$T_1 = h + \sum_{1}^{(256)}(e) + Ch(e,f,g) + K_t^{(256)} + W_t \tag{3.8}$$

$$T_2 = \sum_{0}^{(256)}(a) + Maj(a,b,c) \tag{3.9}$$

$$
\begin{aligned}
h &= g, \\
g &= f, \\
f &= e, \\
e &= d + T_1, \\
d &= c, \\
c &= b, \\
b &= a, \\
a &= T_1 + T_2.
\end{aligned}
$$

- *Intermediate Hash Values*: These values are computed using eight working variables as follows:

$$
\begin{aligned}
H_0^1 &= a + H_0^0, \\
H_1^1 &= b + H_1^0, \\
H_2^1 &= c + H_2^0, \\
H_3^1 &= d + H_3^0, \\
H_4^1 &= e + H_4^0, \\
H_5^1 &= f + H_5^0, \\
H_6^1 &= g + H_6^0, \\
H_7^1 &= h + H_7^0,
\end{aligned}
$$

There are total 64 rounds for working and temporary variables and computation of message schedule words. As the rounds consist of sequential identical operations, this is implemented through the mechanism called Finite State Machine (FSM) in BSV. Further, the careful observation of rounds make their computation possible in parallel so that total number of clock cycles can be optimized. This implementation is explained in fig 3.3. The rule syntax in BSV, is equivalent to always @ syntax in verilog, which will fire at every clock. Further, it can be made conditioned FIRE with the help of if-else conditions, as implemented in figure. The message scheduler W_t, t ϵ [16,63], computed on every alternative clock cycle, with middle clock cycle computes the

required values of functions (s_0, s_1). Whereas the values of functions ($\sum_0$, $\sum_1$) are getting out through clock stealing method. The BSV offers these advantages to implement such time division logic with easy and without bothering developer to think for its hardware. Further, the code can be self-explainable, like some C types of programming languages.

```
//round computation of SHA-256
        rule roundcomp (step == 6);
                rule roundcomp1 (state == 1);
                        //Computing message scheduler (wt) for 16 <= t <= 63
                        w[index] <= ( sig1[index-2] + w[index-7] + sig0[index-15] + w[index-16]);
                        w[index+1] <= ( sig1[index + 1 - 2] + w[index + 1 - 7] + sig0[index + 1 - 15] + w[index + 1 - 16]);

                        sum1e <= funSumOne(e);
                        ch <= funCH(e,f,g);
                        sum0a <= funSumZero(a);
                        maj <= funMaj(a,b,c);

                endrule
                rule roundcomp2 (state == 2);
                        //Computing values of functions parallely sig0, sig1
                        sig1[index] <= funSigmaOne(w[index]);
                        sig1[index + 1] <= funSigmaOne(w[index + 1]);

                        sig0[index] <= funSigmaZero(w[index]);
                        sig0[index + 1] <= funSigmaZero(w[index + 1]);

                        //Computing for two temporary words T1 and T2
                        t1 <= h + sum1e + ch + k.sub(counter) + w[counter];
                        t2 <= sum0a + maj;

                        index <= index + 2;
                        state <= 3;
                endrule
                rule roundcomp3 (state == 3);
                        //Computing message scheduler (wt) for 16 <= t <= 63
                        w[index] <= ( sig1[index-2] + w[index-7] + sig0[index-15] + w[index-16]);
                        w[index+1] <= ( sig1[index + 1 - 2] + w[index + 1 - 7] + sig0[index + 1 - 15] + w[index + 1 - 16]);

                        //Assigning values for next stage calculation to a to h.
                        a <= t1 + t2;
                        e <= d + t1;
                        h <= g; g <= f; f <= e;
                        b <= a; c <= b; d <= c;
                endrule
        endrule
```

Figure 3.3: Implementation of Round Computation in BSV carried out in thesis

The intermediate digest module outputs the next stage hash values as explained above with addition of eight working variables and initial hash values. This is hash values of current padded block. If the next padded block is present then this intermediate digest value will become the initial hash value for next padded block. With this, the design is having modular approach and can be re-usable for any number of padded blocks.

The same architecture can be utilized with changes discussed previously to obtain SHA-512 and subsequently, SHA-224 and SHA-384 can be achieved with trimming the result of SHA-256 and SHA-512, respectively.

3.3 Comparative Analysis

As discussed in beginning of the chapter that the related work can not be fitted, as it is, to carry out the integration with RISC V core, therefore new implementation of SHA-2 was carried out in BSV language and also thesis has proposed architecture as shown in figure 3.2. This includes the add-on modules such as fragmentation-reassembly, interfaces for flow and computation control like load for command to accept the input message, start for command to start algorithm, finish for command to inform external world for completion of algorithm and fetch for command to provide the intermediate or final digest.

Thesis has carried out the implementation of SHA-224, SHA-256, SHA-384 and SHA-512 using BSV language. The BSV has generated verilog files for each on request. These verilog files are synthesized on various platforms. These platforms includes Xilinx FPGAs Virtex-7 family (device-xc7vx330tffg1157-3)[14] and Artix-7 AC701 evaluation platform (xc7a200tfbg676-2)[15]. The Vivado 2016.2 development environment[16] was utilized. Further, the implementation was also synthesized on Stratix V Family (device-5SGXEA7H2F35C3) [17] from Altera FPGAs. In this case, Quartus II 64bit[18] as development environment was used. The both device follows the 28nm technology.

The results of above synthesis are explained in tabular format table numbers3.1, 3.2 and 3.3.

Table 3.1: Synthesis result on Virtex-7 Family

Virtex-7 Family (device-xc7vx330tffg1157-3)			
SHA-2 Variants	CLB	FF	$F_{MAX}(MHz)$
SHA-224	7068	7255	123
SHA-256	7477	7176	123
SHA-384	25129	16525	111
SHA-512	25941	16784	109

In case of Altera, the area is described in terms of Adaptive Logic Module (ALM), which is basic building block. However, in case of Xilinx, the area is described in terms of Configurable Logic Block (CLB). The terminology is different but they have similar interpretation. The FF tends for Flip-Flop.

Table 3.2: Synthesis result on Artix-7 Evaluation platform

Artix-7 Evaluation platform			
SHA-2 Variants	CLB	FF	$F_{MAX}(MHz)$
SHA-224	7069	7255	99
SHA-256	7471	7176	98
SHA-384	25954	16516	84
SHA-512	25480	16781	81

Table 3.3: Synthesis result on Stratix-V Family

Stratix-V Family (device-5SGXEA7H2F35C3)			
SHA-2 Variants	ALM	FF	$F_{MAX}(MHz)$
SHA-224	7736	8098	102
SHA-256	7774	8118	100
SHA-384	18943	17836	71
SHA-512	19075	18172	70

The estimated power consumption of above implementation is in the range of 0.16 to 0.19 watt.

The round computation is more for SHA-384 and SHA-512 compare to SHA-224 and SHA-256 therefore the number of resources utilized can be justified. Further, the operation on 64 bits consume more cells in case of SHA-384 and SHA-512.

Constraint for the operating frequency was set to 100MHz, however, it can be seen that it varies over different FPGAs family. Our proposed design can operate at maximum frequency 123 MHz.

In summary, The SHA-224 and SHA-256 acquire almost the same area and can be operated at same F_{max} which is easy to justify as they have same steps of algorithm. Further, The SHA-384 and SHA-512 also have same steps and therefore they also acquire almost same area and can be operated at same F_{max}.

Chapter 4: Integration with RISC-V core

The chapter explains the final part of the thesis. The selection of integration strategy with host processor is finalized in consideration of open source and open standard mechanisms. This has advantages from developer side, as he/she can get help from open and free market and at the same time, developer is adding the value to the existing open and free source.

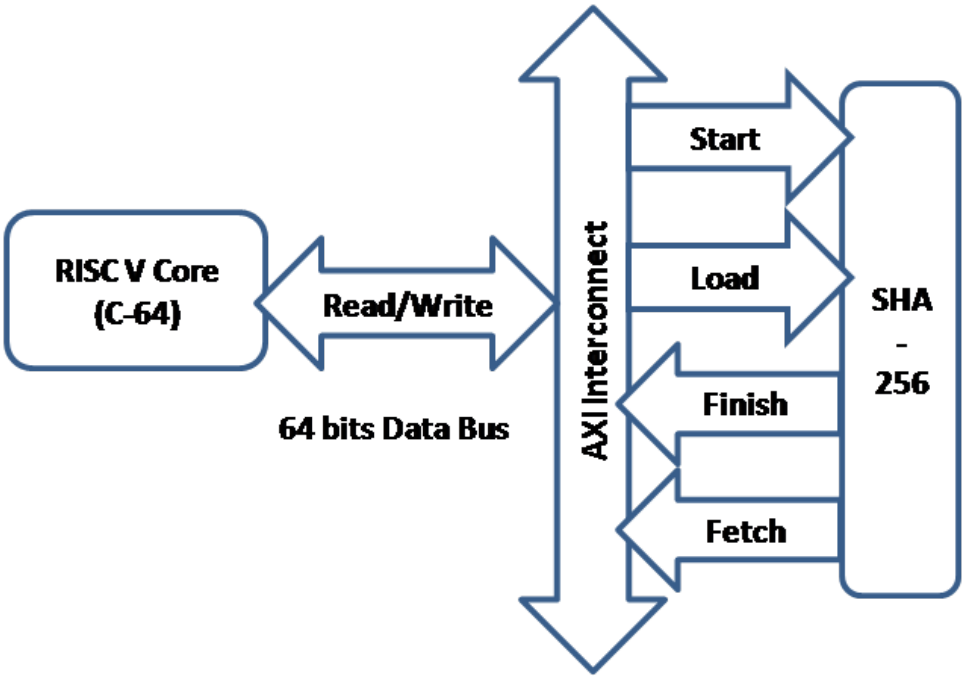


Figure 4.1: Identified strategy for integration with RISC V core

4.1 Description of identified strategy

The integration of SHA-2 FPGA implementation with host-processor, in our case, RISCV core, C-64, is shown in figure 4.1. The host-processor, C-64 is the SHAKTI processor and has open source for its ISA code therefore it is possible to compile and integrate interested peripheral in SoC designs. The integration has followed the memory map mechanism[19], where, each peripheral is assigned with particular, pre-defined memory address. So that whenever processor wants to read/write it uses that memory address. The selection of Advanced eXtensible Interface (AXI) interconnect is from its popular usage in ARM development [20]. Further, It is open standard, which specifies the connection and management of functional blocks in SoC designs. To support these connections, AXI has five different channels: Read and Write Address Channel, Read and Write Data Channel and lastly Write Response Channel. [21].These channels provide the communication path between AXI Master - AXI Slave and AXI Monitor. AXI Master generates the address and control for particular peripheral. This will decoded by AXI Slave and enable that particular peripheral for read operation. For Write operation, AXI Master generates the address and control for particular peripheral, which will decoded by AXI Slave and enable that peripheral for write operation. Here after write operation, there is write response to acknowledge the previous write, generated by AXI Slave.

Based on selection of AXI bus interconnect, System On Chip (SoC) approach is processed. Here the processor along with its peripherals and the SHA-2 implementation will be fused into single chip. SHA-2 implementation is integrated as one of the peripherals, which helps in minimizing the transfer time with processor.

Further, the AXI interface is integrated with SHA-256 implementation in BSV. This helps SHA-256 to place request and response to/from the processor. Similarly, the AXI interface is integrated with processor code, to help processor in communicating with SHA-256.

4.1.1 Customized Protocol between RISC V and SHA-2

Given that data bus size of the RISC V Core, C-64, is 64 bits. We need to write the fragmentation module. In the same way, the reassembly

module is also implemented along with SHA-256 FPGA implementation. The Protocol as follows:

- *Step-1*: For given input message length l, padding process is applied. It will result in to multiple blocks of 512 bits. The pseudo code is explained as below.

Algorithm 1 Padding Block

```
1: procedure PADDING(m, l)        ▷ The input message m with length l
2:     while r ≠ n · 512 do       ▷ Until r is multiple of 512bits
3:         r ← m||'b1             ▷ append bit 1
4:         k ← r mod 512          ▷ finding k
5:         r ← r||k('b0)          ▷ appending k times number of bit 0
6:         r ← r||'d(l)           ▷ appending 64 bits, representing l
7:     end while
8:     return r                   ▷ Padded block
9: end procedure
```

- *Step-2*: For each block of 512bits of padded message, fragmentation is applied. It will result into eight, small block of 64 bits, which can be loaded to SHA-256 module.
- *Step-3*: On completion of step-2, start register will be loaded with value 1. This will enable processing of SHA-256 module.
- *Step-4*: Once computation of intermediate digest for given 512 bits is completed, SHA-256 module writes the finish register with value 1.
- *Step-5*: The finish register will be polled by host-processor, once its value is equal to 1. Processor initiates reading of intermediate digest with the help of Fetch register. Here once again, the fragmentation logic is applied for seamless transaction.
- *Step-6*: In case of more than one padded block, step- 2 to 5 is repeated.

The customized protocol explains the integration with SHA-256 implementation with RISC V core. The same strategy is followed with SHA-224, SHA-384 and SHA-512.

The above protocol is realized by implementation of code in standard C programming language. Further, to execute it on RISC V Core requires the cross compilation. The RISC V compilation tool chain [22] is followed to carry out the executable. In addition to this, to support the any length of input message, the fragmentation and reassembly for input message and fetching of intermediate digest are written in standard C programming language and executed on processor.

4.2 Result

This section evaluates the strategy based on area, power and performance overheads. It is explained with the help of table shown in4.1.

The results presented in previous chapter provide us the confidence that our implementation can synthesize and we can go ahead for tap out procedure. This is because of most important advantage of BSV which guarantees that almost all compiled code can synthesize. This can be appreciated using Encounter RTL Compiler(RC) [23] tool from Cadance. The table shows the result provided by RC tool. The result is compiled using 40nm technology library.

Table 4.1: Synthesis result using RTL Compiler

At 1 GHz			
SHA-2 Variants	Area	Power(mW)	Performance (MHz)
SHA-224	081002	135	410
SHA-256	082040	144	415
SHA-384	237259	646	376
SHA-512	241052	653	383

The table shows that the area, power and performance (F_{Max}) for SHA-224 and SHA-256 are almost same, which is because of identical structure of their algorithm. Similarly, SHA-384 and SHA-512 has almost same power, area and performance overheads. The proposed design can operate at maximum frequency of 415 MHz. The power consumed by proposed design is 653 mW at max. The area described in table represents the total number of gates utilized in design.

Chapter 5: Conclusion

In this thesis, successful FPGA implementation of SHA-2 was carried out using recently introduced Hardware Description Language, Bluespec System Verilog and with the extensive survey of related work. This implementation was synthesized over various platforms, Virtex-7, Stratix V and Artix-7 evaluation board. The objective of thesis, to integrate SHA-2 implementation with RISC V Core such as SHAKTI C-64, was successfully carried out. Thesis had identified the integration strategy with the help of AXI interconnect in SoC format with host processor and it was realized successfully. Further, it evaluated with power, area and performance overheads. In future, thesis suggests the further improvement in implementation and integration work.

Bibliography

[1] http://rise.cse.iitm.ac.in/rise1/projects/shakti.html

[2] National Institute of Standards and Technology (1994) Secure hash standard. In: Federal Information Processing Standards Publication 180–1, April 1994

[3] N. Sklavos and O. Koufopavlou, "On the hardware implementations of the SHA-2 (256, 384, 512) hash functions," Circuits and Systems, 2003. ISCAS '03

[4] Sklavos, Nicolas, and Odysseas Koufopavlou. "Implementation of the SHA-2 hash family standard using FPGAs." The Journal of Supercomputing 31.3 (2005): 227-248

[5] Kurt K. Ting, Steve C.L. Yuen, K. H. Lee, and Philip H.W. Leong, "An FPGA based SHA-256 Processor", Field-Programmable Logic and Applications: Reconfigurable Computing Is Going Mainstream: 12th International Conference, FPL 2002 Montpellier, France, September 2–4, 2002 Proceedings.

[6] Bluespec System Verilog, http://bluespec.com/

[7] https://cloudtweaks.com/2012/08/advantages-and-disadvantages-of-open-source/.

[8] http://www.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html.

[9] https://www.slideshare.net/VishakhaAgarwal4/secure-hash-algorithm

[10] http://wiki.bluespec.com/Home/Getting-Started/Tutorials

[11] S. Dominikus, "A Hardware Implementation of MD4- Family Hash Algorithms", proc. of IEEE International Conference on Electronics Circuits and Systems (ICECS'OZ), Vol. Ill, pp.1143-1146, Croatia, 2002.

[12] P.H.W. Leong, M.P. Leong, O.Y.H. Cheung, T. Tung, C.M. Kwok, M.Y. Wong, and K.H. Lee, "Pilchard — a reconfigurable computing platform with memory slot interface", IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM), 2001.

[13] https://csrc.nist.gov/csrc/media/publications/fips/180/4/final/documents/fips180-4-draft-aug2014.pdf.

[14] https://www.xilinx.com/products/silicon-devices/fpga/virtex-7.html.

[15] https://www.xilinx.com/products/boards-and-kits/ek-a7-ac701-g.html.

[16] https://www.xilinx.com/support/documentation/sw_manuals/xilinx2016_2/ug973-vivado-release-notes-install-license.pdf.

[17] https://www.altera.com/products/fpga/stratix-series/stratix-v/overview.html.

[18] https://www.element14.com/community/docs/DOC-40098/l/altera-introduction-to-the-quartus-ii-software.

[19] https://www.ibm.com/support/knowledgecenter/en/ssw_aix_72/com.ibm.aix.genprogc/understanding_mem_mapping.htm.

[20] http://www.gstitt.ece.ufl.edu/courses/fall15/eel4720-5721/labs/refs/AXI4-specification.pdf.

[21] https://www.slideshare.net/vintrainvlsitraining/axi-14777962.

[22] https://github.com/riscv/riscv-gnu-toolchain.

[23] https://www.csee.umbc.edu/ tinoosh/cmpe641/tutorials/rc/rc-flow.